Model-Based vs. Model-Free Reinforcement Learning: Value Iteration and Q-Learning on the Stochastic Windy Gridworld

Tanzila Khan
Roll - 25

1 Objective

This lab compares a model-based method (Value Iteration) with a model-free method (Q-Learning) on the same Markov Decision Process and examines how their hyperparameters behave. The environment is the **Stochastic Windy Gridworld**, a grid-world navigation task with a column-dependent, stochastic crosswind, implemented as a custom Gymnasium environment.

Windy Gridworld is multi-step, delayed-reward, and stochastic through a column-dependent wind. This combination is what lets γ change the optimal *policy* itself here, rather than only the speed of convergence (Section 8.1). Most states (64 of 69) also have a clearly best action with a real margin over the runner-up (Section 7), so Q-Learning is expected to recover Value Iteration's policy on almost all of them.

2 Environment

A 7×10 grid. The agent starts at $S = (3, 0)$ (row, column) and must reach the goal $G = (3, 7)$. Four actions are available: UP, DOWN, LEFT, RIGHT, each moving one cell (clipped at the grid edge). The wind strength used for a move is read from the column the agent *occupies before* the move (the standard convention), and the resulting row shift is applied to the landing column:

column	0	1	2	3	4	5	6	7	8	9
mean wind	0	0	0	1	1	1	2	2	1	0

In a windy column (mean wind $w > 0$), the *actual* push each time the agent passes through is stochastic: $w - 1$, w, or $w + 1$, each with probability $1/3$; calm columns push deterministically by 0. All positions are clipped to the grid.

- **State:** `(row, col)`, a genuine Markov state, since the wind at a column is drawn i.i.d. every time the agent passes through it. The model has 69 non-terminal states (the goal is absorbing).
- **Actions:** `0=UP, 1=DOWN, 2=LEFT, 3=RIGHT` (`Discrete(4)`).
- **Reward:** -1 on every transition, including the one that reaches the goal, which ends the episode.

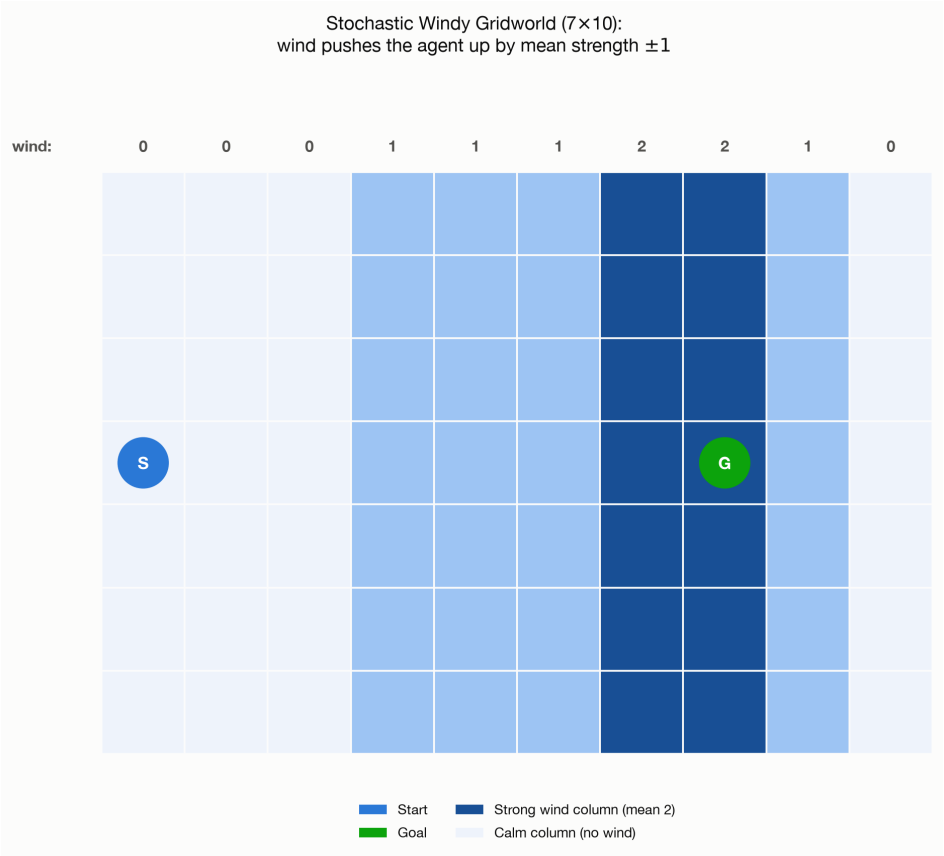


Figure 1: The Stochastic Windy Gridworld. Darker columns carry stronger mean wind; the wind's actual strength on any single pass varies by ± 1.

3 Theoretical Grounding

Both algorithms compute the same object. Under the **principle of maximum expected utility (MEU)**, a rational agent chooses the action maximising

$$EU(a) = \sum_{s'} P(\text{RESULT}(a) = s')\, U(s').$$

Value Iteration computes exactly this at each state from the derived wind model. The **Markov property** (Section 2) makes the **Bellman equation**

$$U(s) = \max_a \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma\, U(s')\big]$$

well defined; Value Iteration solves it by repeated Bellman updates, Q-Learning estimates the same right-hand side online with learning rate α, sampling transitions from the real environment instead of summing over a known model. In the course's classification, both methods are *value-based*, and Q-Learning is *off-policy*: its update bootstraps off $\max_a Q(s', a)$ regardless of the exploratory action actually taken.

Why γ matters here. With reward -1 per step until an absorbing goal, the discounted return from a state under a fixed number of remaining steps T is $-\frac{1-\gamma^T}{1-\gamma}$, which as a function of the step

count T is decreasing and convex (equivalently, the accumulated cost $\frac{1-\gamma^T}{1-\gamma}$ is increasing and concave in T). In a *deterministic* shortest-path problem, each fixed policy induces a fixed completion time T, so any $\gamma \in (0,1)$ ranks policies identically: γ only has to be strictly less than 1. Here T is a *random variable* (the wind), so a policy's value is $\mathbb{E}\left[-\frac{1-\gamma^T}{1-\gamma}\right] = -\frac{1-\mathbb{E}[\gamma^T]}{1-\gamma}$, which by Jensen's inequality depends on the whole distribution of T through $\mathbb{E}[\gamma^T]$, not only its mean. The value of γ sets how heavily the tail of the step-count distribution is weighted and can change which policy is optimal. Section 8.1 shows this is not a theoretical curiosity: at $\gamma = 0.7$ the greedy policy is different from $\gamma \geq 0.9$'s, not merely slower to compute.

4 Deriving the Transition Model

Because Windy Gridworld is a custom environment, Gymnasium exposes no transition table, so the model is derived analytically from the wind's known probabilities. From state `(r, c)` taking action a: the deterministic part of the move gives an intermediate cell `(r1, c1)` (clipped to the grid). Wind strength is determined by the pre-action column `c`, not the landing column `c1`: if column `c` is calm, the wind push is deterministically 0; if it carries mean wind $w > 0$, the push is $w-1$, w, or $w+1$ each with probability 1/3. That row shift is then applied to the landing column: the next state is `(r1 - push, c1)`, clipped to the grid, with reward -1 and the episode ending if that cell is the goal.

5 Value Iteration

Algorithm. Value Iteration turns the Bellman optimality equation (Section 3) into a synchronous update rule, applied to every state until the value function stops changing appreciably:

1. **Initialise** $V_0(s) = 0$ for every non-terminal state s; $V(s_{\text{goal}}) = 0$, fixed, since the goal is absorbing.
2. **Repeat** for sweep $k = 1, 2, \ldots$:
   (a) For every non-terminal state s and every action a, compute the action value under the current estimate, using the transition model derived in Section 4:
   $$Q_k(s,a) = \sum_{s'} P(s' \mid s,a)\big[R(s,a,s') + \gamma\, V_{k-1}(s')\big]$$
   (b) Update the state value greedily: $V_k(s) = \max_a Q_k(s,a)$.
   (c) Track the largest change across all states, $\Delta_k = \max_s |V_k(s) - V_{k-1}(s)|$.
3. **Stop** once $\Delta_k < \theta$.
4. **Extract** the greedy policy from the converged V^*:
   $$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s,a)\big[R(s,a,s') + \gamma\, V^*(s')\big]$$

Value Iteration was run with $\gamma = 0.99$ and threshold $\theta = 10^{-10}$; it converges in **116 sweeps** (Figure **??**). Rolled out from the start for 5,000 episodes (evaluation seed 1), the greedy policy reaches the goal every time, in 20.53 steps on average. Figure **??** shows the resulting value function:

$V^*(s) = -$(expected discounted steps to the goal), lowest (most negative) far from the goal and in the strong-wind columns, where the stochastic overshoot is costliest to correct.

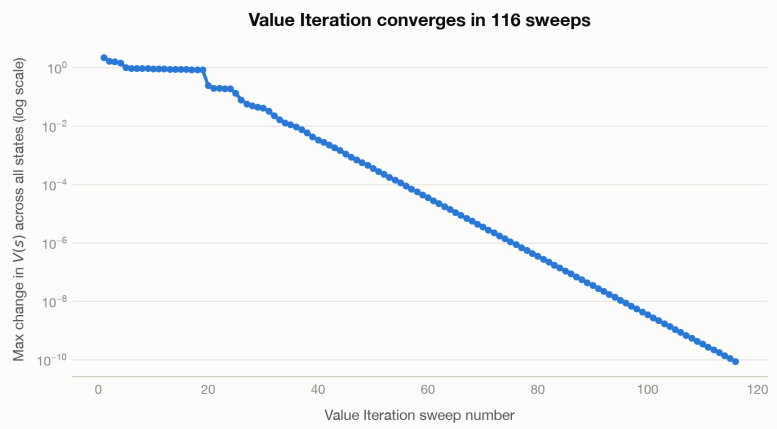


Figure 2: Value Iteration convergence.

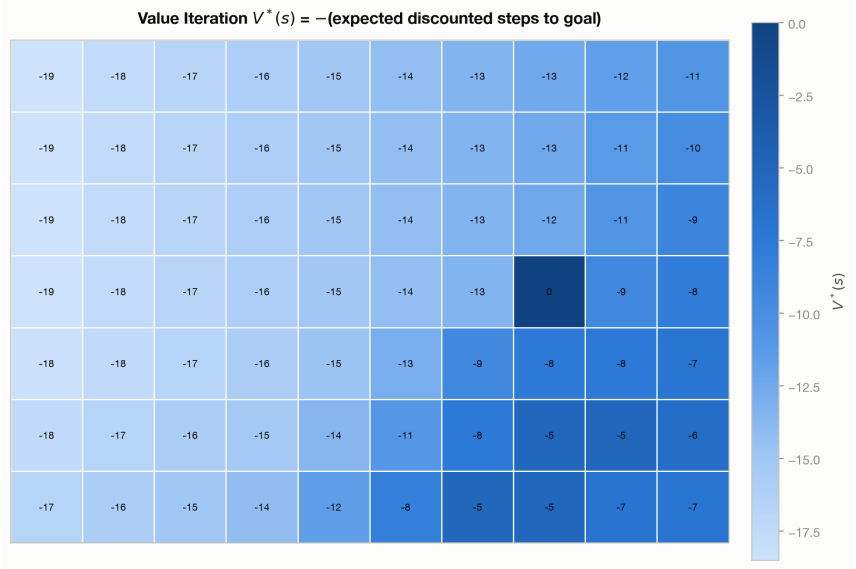


Figure 3: $V^*(\text{row}, \text{col})$ over the grid.

6 Q-Learning

Algorithm. Q-Learning estimates the same Bellman optimality equation without a transition model, by sampling transitions from the real environment and bootstrapping off its own current estimate:

1. **Initialise** $Q(s, a) = 0$ for every state-action pair; set the exploration rate $\varepsilon = \varepsilon_0$.
2. **For each episode:**
   (a) Initialise the state s at the fixed start.

(b) **Repeat** for each step until s is terminal:

i. Choose action a from s using an ε-greedy policy over $Q(s, \cdot)$: the greedy action $\arg\max_a Q(s, a)$ with probability $1 - \varepsilon$, a uniformly random action otherwise.

ii. Take action a via `env.step(a)`; observe reward r and next state s'.

iii. Update the estimate toward the sampled Bellman target:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \Big]$$

iv. $s \leftarrow s'$.

(c) Decay ε toward its floor.

3. **Extract** the greedy policy: $\pi(s) = \arg\max_a Q(s, a)$.

Q-Learning was trained against the real environment (`env.step()` only) for a single run (seed 42), with $\alpha = 0.1$, $\gamma = 0.99$, and ε-greedy exploration decaying multiplicatively as $\varepsilon_t = \max(0.02,\ 1.0 \times 0.9999^{\,t})$ per episode t, over 50,000 episodes. No exploring starts were used: the decaying-ε schedule provided sufficient empirical coverage for this run, and all states were visited during training. Section 8.2 shows the learner needs on the order of 10^4 episodes to approach Value Iteration's optimum, because every early attempt to reach the goal must first survive the wind's variance. Rolled out greedily for 5,000 episodes (evaluation seed 3, a separate batch from Value Iteration's), the learned policy also reaches the goal every time, in 20.53 steps on average. Both methods produced approximately 20.53 steps on separate Monte Carlo evaluations; the small numerical difference should not be interpreted as a policy-performance difference (Figure **??**).

7 Comparing and Validating the Two Policies

7.1 Performance parity

Both methods solve the task essentially perfectly: 100% success, and average path length differs only within ordinary Monte Carlo evaluation noise between the two methods' separate rollout batches (Figure **??**). Performance alone does not tell the full story; the distinguishing result is the *policy* comparison below.

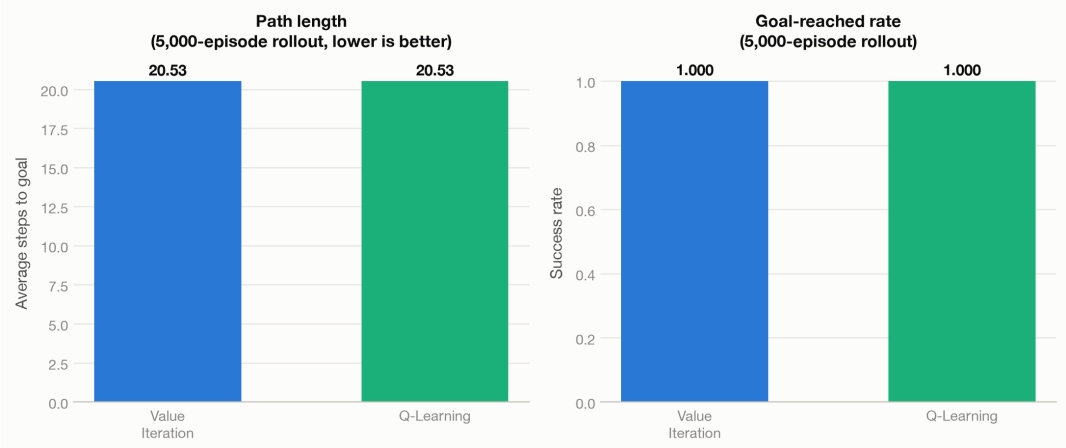


Figure 4: Performance parity: path length and success rate, 5,000-episode rollouts.

7.2 The policies agree almost everywhere: the optimum is not flat

Q-Learning's greedy policy agrees with Value Iteration's exact policy on **61 of 69** states (88.4%), and every state is visited at least once over 50,000 training episodes (Figure **??**). Figure **??** shows why: the median best-vs-second-best action margin across the MDP is 0.83, and 64 of the 69 states have a margin of at least 0.1. The margin only collapses toward zero at a small cluster of low-margin states where several directions are near-equivalent because there is no wind there to break the tie. With a clearly best action almost everywhere, a sampling-based learner given enough episodes converges to the same action on the vast majority of states; the eight disagreements left are concentrated at that near-tied cluster and at states a near-optimal policy rarely visits, which are consequently the last to be learned (Figure **??**).

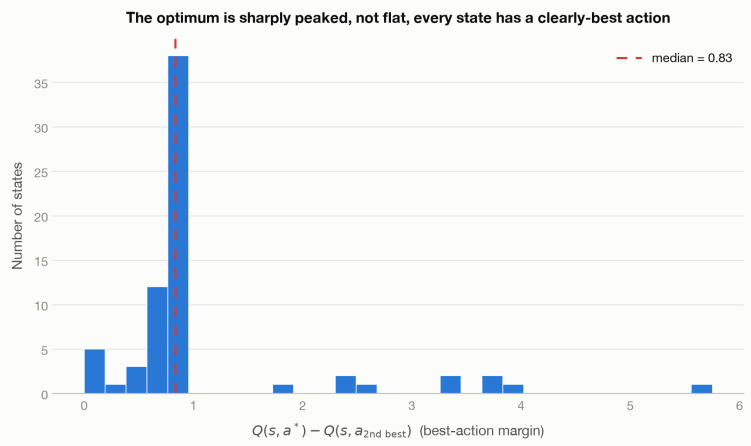


Figure 5: Distribution of the best-action margin across all 69 states: sharply peaked well above zero.

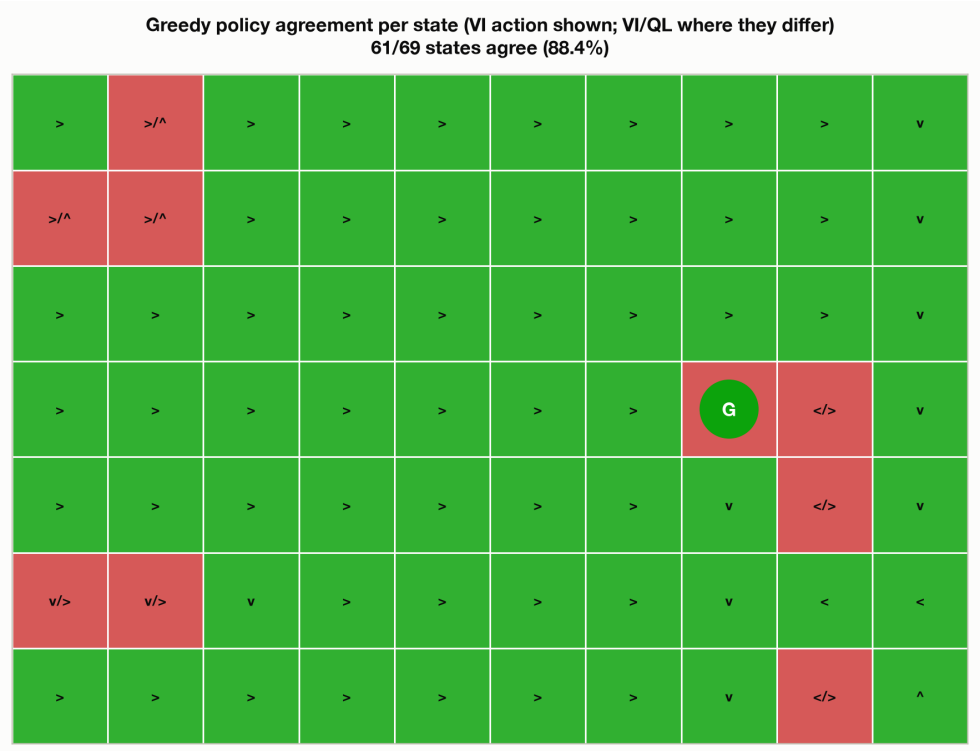


Figure 6: Per-state greedy action: Value Iteration's action is shown; where Q-Learning disagrees, both are shown as VI/QL. Green = agreement, red = disagreement among non-terminal cells; the absorbing goal (marked G) is excluded from the 69 policy states and is not a disagreement despite its cell shading.

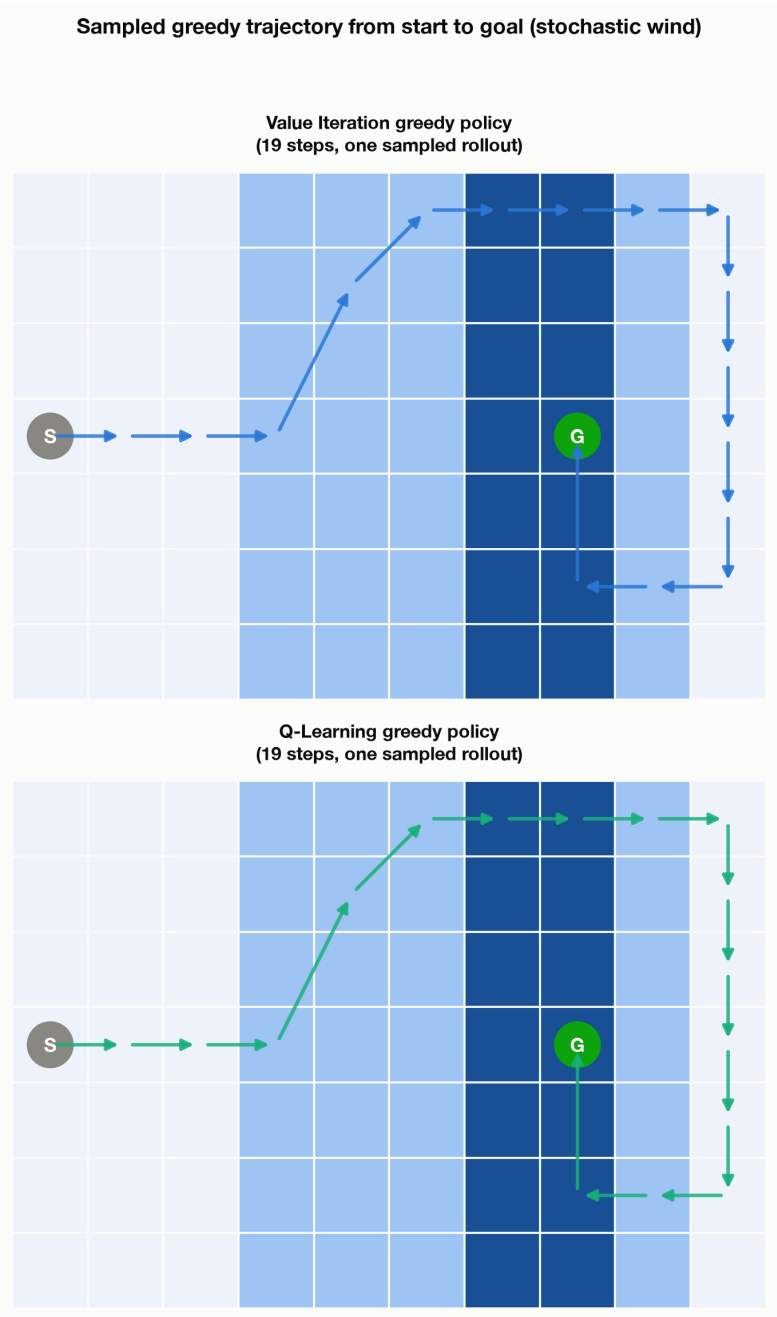


Figure 7: A single sampled greedy rollout for each policy, from the same start under the same stochastic wind seed; both take the same route and the same number of steps on this sample.

8 Hyperparameter Experiments

8.1 Discount factor changes the policy, not only convergence speed

Figure **??** sweeps $\gamma \in \{0.7, 0.9, 0.95, 0.99, 1.0\}$. For $\gamma \geq 0.95$ the greedy average path length is flat at 20.37 steps, effectively the same policy. Lowering γ further changes the policy more substantially: at $\gamma = 0.9$, 6 of 69 states switch their optimal action and average path length rises to 21.37 steps;

at $\gamma = 0.7$, 13 of 69 states differ and path length rises to 27.75 steps. This matches the prediction of Section 3: because a smaller γ discounts distant steps much more heavily, the objective is no longer a good proxy for raw expected step count once the wind's stochastic branching is in play, and Value Iteration finds a policy that is worse in expectation under the undiscounted metric even though it is optimal for the smaller-γ objective it was asked to solve. This effect would not occur in the corresponding deterministic constant-step-cost shortest-path formulation; Windy Gridworld's real stochasticity is what produces it. (The sweep's $\gamma = 1.0$ point is undiscounted: the return there is interpreted by its limit as $-T$ and does not inherit the usual discounted Bellman-contraction guarantee assumed for $\gamma \in (0, 1)$ elsewhere in this section.)

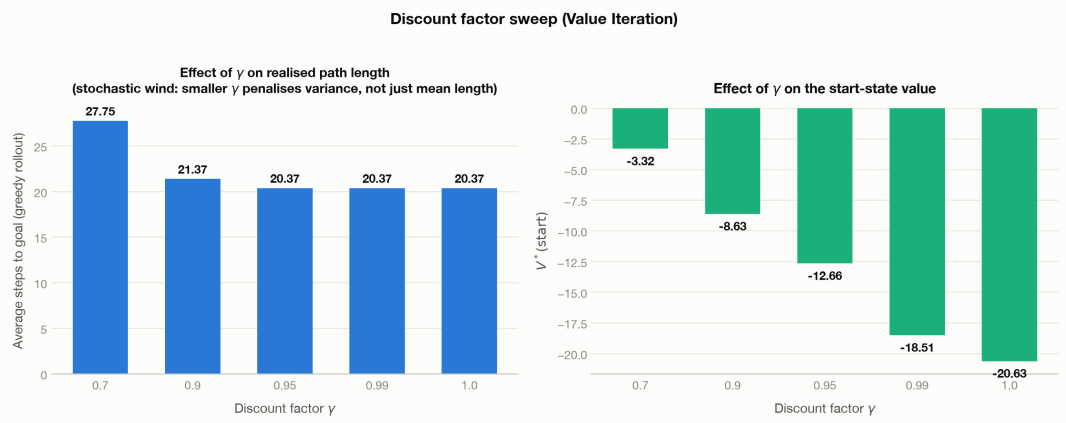


Figure 8: Discount factor sweep: realised path length is flat for $\gamma \geq 0.95$ but rises as γ drops to 0.9 and 0.7, where 6 and 13 states' optimal actions respectively actually change.

8.2 Stopping threshold: cost without benefit, here

Value Iteration's other named hyperparameter is the stopping threshold θ: the sweep terminates once the maximum change in V across all states drops below θ. Figure **??** sweeps six values of θ from 10^{-2} down to 10^{-12} at $\gamma = 0.99$. The cost side follows the Bellman contraction directly: tightening θ by two orders of magnitude costs a roughly fixed number of extra sweeps (36, 56, 76, 96, 116, 136), an arithmetic rather than exponential progression, since each sweep shrinks the remaining error by the fixed factor $\gamma = 0.99$. The benefit side is flat: comparing every θ's greedy policy against the tightest run ($\theta = 10^{-12}$) gives 100% agreement at every tested value, including the loosest $\theta = 10^{-2}$. In this MDP the greedy policy was unchanged for every tested threshold, including $\theta = 10^{-2}$. Unlike γ, which changes *which* policy is optimal (Section 8.1), θ only trades sweeps for numerical precision that this task does not need; the default $\theta = 10^{-10}$ used elsewhere in this report is more than tight enough, though a much looser value would have given the identical policy for a third of the sweeps.

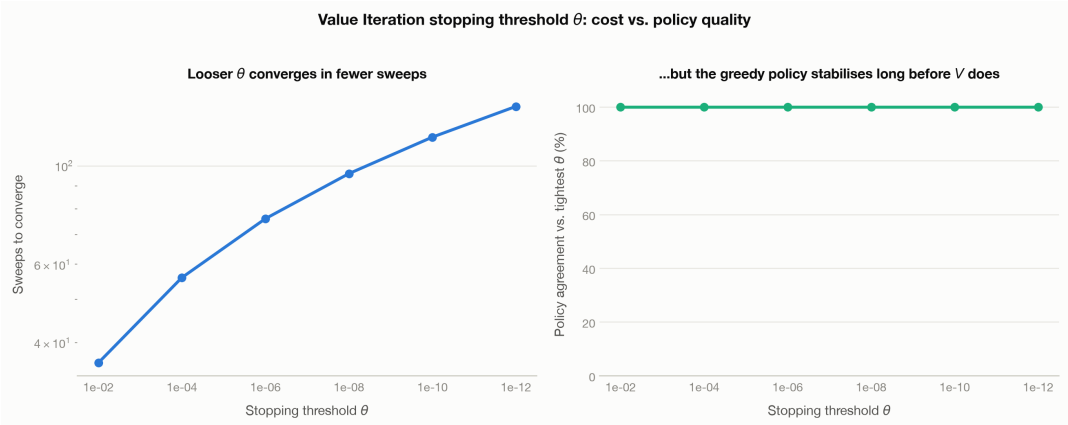


Figure 9: Stopping-threshold sweep: sweeps-to-converge rises steadily as θ tightens (left), while the resulting greedy policy is identical to the tightest-θ policy at every setting tested (right).

8.3 Learning rate and exploration decay

Figure **??** sweeps Q-Learning's learning rate over the exact set $\alpha \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$. All settings converge toward Value Iteration's optimum; smaller α (0.02–0.1) converges to a slightly lower, steadier plateau, while larger α (0.3–0.5) is noisier and settles a little above the optimum, the familiar bias/variance trade-off of the running average that a Q-value update performs. The exploration-decay sweep (Figure **??**) uses the multiplicative schedule $\varepsilon_t = \max(0.02, \varepsilon_0 d^t)$ with $d \in \{0.999, 0.9999, 0.99995\}$, and shows the same overall pattern: decaying ε too fast ($d = 0.999$) starves the agent of the exploration it needs to discover the shorter route around the wind, while a slower decay ($d = 0.99995$) reaches the optimum in more episodes but more reliably. Both sweeps train for the same 50,000 episodes as the main Q-Learning run above, each setting as a single run with seed 42 rather than an average over multiple seeds, and both figures plot a 100-episode moving average of steps to goal.

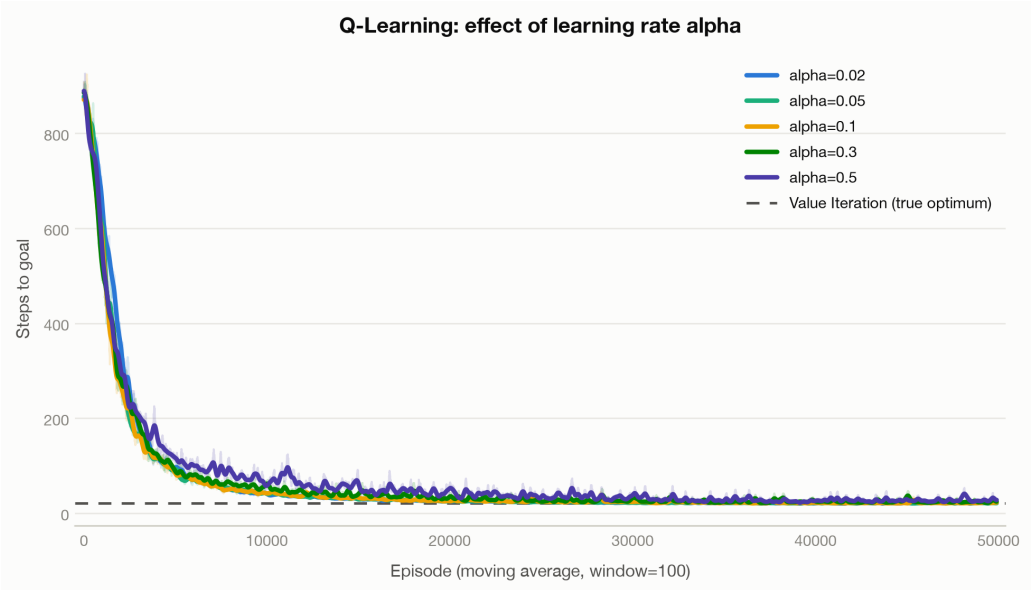


Figure 10: Effect of the learning rate on Q-Learning's convergence toward Value Iteration's optimum (lower is better).

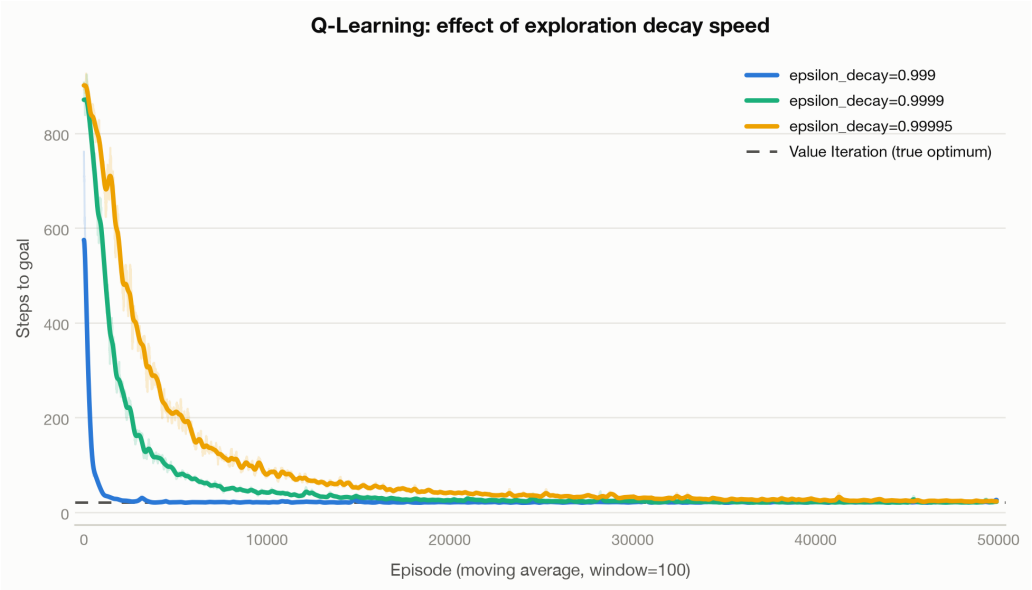


Figure 11: Effect of the exploration-decay rate on Q-Learning's convergence toward Value Iteration's optimum (lower is better).

9 Model-Based vs. Model-Free

Table 1: Model-based vs. model-free on the Stochastic Windy Gridworld.

	Value Iteration (model-based)	Q-Learning (model-free)
Needs a transition model	Yes: derived from the wind's known probabilities	No: only `env.step()`
Environment interactions	0	50,000 episodes (∼20 steps each)
Recovers the exact policy	Yes (exact expectations)	Yes, on 88.4% of states; the optimum is not flat, so sampling suffices
Rollout performance	20.53 avg. steps	20.53 avg. steps on a separate evaluation batch
Sensitive to γ's exact value	Yes: changes the policy under real stochasticity	Not separately tested; Q-Learning optimizes the same discounted objective
Sensitive to θ's exact value	No: identical policy across all tested θ (Section 8.2)	N/A: has no stopping threshold

With a stochastic environment and a sharply-peaked (non-flat) optimum, Q-Learning matches Value Iteration's performance and recovers nearly the same policy, state by state, with no model and no access to the wind's probabilities. The remaining disagreements are the rarely-visited states any sample-based method would learn last. The fundamental trade-off between the two methods is a model bought with derivation effort and zero environment interaction, versus tens of thousands of real episodes needed to learn from scratch.

10 Conclusion

A transition model for the Stochastic Windy Gridworld was derived analytically from the wind's known probabilities and solved exactly with Value Iteration in 116 sweeps, reaching the goal in 20.53 steps on average. Q-Learning, trained without any model for 50,000 episodes, produced approximately the same rollout performance (20.53 steps, on a separate evaluation batch) and recovered the *same* greedy policy on 88.4% of states. This follows from the MDP's mostly sharply-peaked optimum: 64 of 69 states have an action-value margin of at least 0.1. The discount-factor sweep showed that, because the wind makes the environment stochastic, γ does more than guarantee convergence: dropping γ from 0.99 to 0.9 or 0.7 changes the optimal action at 6 and 13 states respectively. The stopping threshold θ, Value Iteration's other named hyperparameter, only traded sweeps for numerical precision this task did not need: the greedy policy was identical across six orders of magnitude of θ. Windy Gridworld's combination of genuine stochasticity and a sharply-peaked optimum is exactly what lets model-free learning recover both the model-based performance and the model-based policy.